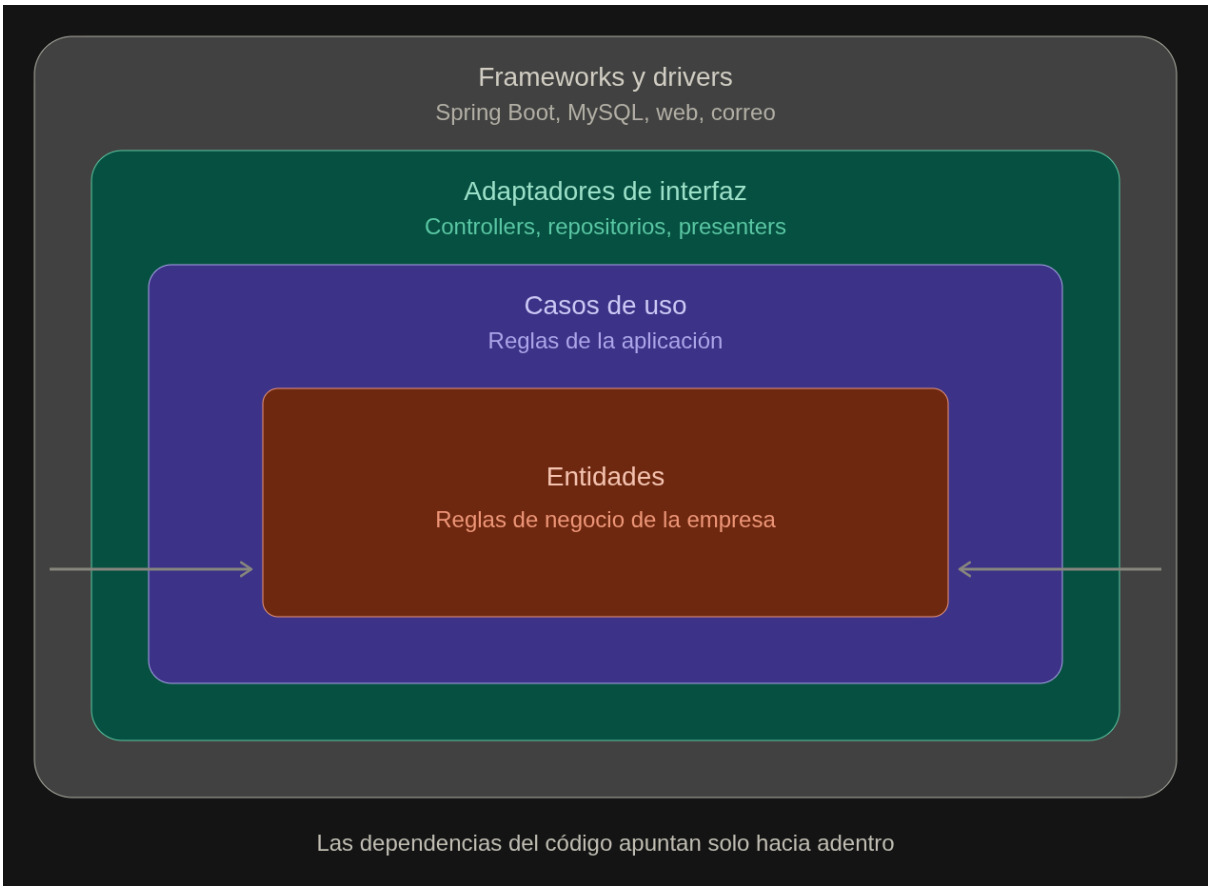
a) Investigación:

➢ ¿Qué significa que una arquitectura sea limpia “clean architecture”? Grafique la disposición de capas y explique su funcionamiento. Indique sus ventajas y desventajas. ¿Qué diferencia existe entre “clean architecture” y la arquitectura por capas tradicional? Mostrar con un gráfico la diferencia entre ambas.

La arquitectura limpia es un paradigma de diseño de software que organiza el sistema en círculos separando las responsabilidades para que las reglas del negocio no dependan de nada externo. Su meta es crear sistemas que sean:

- Independientes de frameworks y librerías externas.
- Altamente comprobables de manera aislada.
- Independientes de la interfaz de usuario.
- Independientes de la base de datos.
- Independientes de cualquier agente externo al negocio.



Capas:

1. Entidades:

    - Están en el núcleo. Encapsulan las reglas de negocio críticas de la empresa. Pueden ser objetos con lógica de negocio pura que no cambian sin importar cómo sea modificada la aplicación.
2. Casos de Uso:
    - Se encargan del flujo de datos desde y hacia las entidades. Contienen la lógica específica de la aplicación. No les interesa la interfaz web.
3. Adaptadores de Interfaces:
    - Son los encargados de convertir los datos del formato que más convenga de los casos de uso y entidades al de agencias externas. En este nivel se encuentran los controladores, gateways y presenters.
4. Frameworks & Drivers:
    - Es la capa más externa, compuesta por herramientas y detalles técnicos: bases de datos, frameworks web (como Spring Boot), servidores HTTP, UI, etc. Son "detalles" que se pueden cambiar fácilmente sin afectar al núcleo de la aplicación.

ventajas

- Mantenibilidad y Escalabilidad: Al estar desacoplado, modificar una base de datos o cambiar de framework no rompe la lógica de negocio.
- Testabilidad excelente: Los casos de uso y entidades se pueden probar unitariamente mediante código puro, sin levantar servidores ni bases de datos.
- Independencia tecnológica: Permite retrasar decisiones de diseño tecnológicas (qué base de datos usar, qué ORM, etc.).

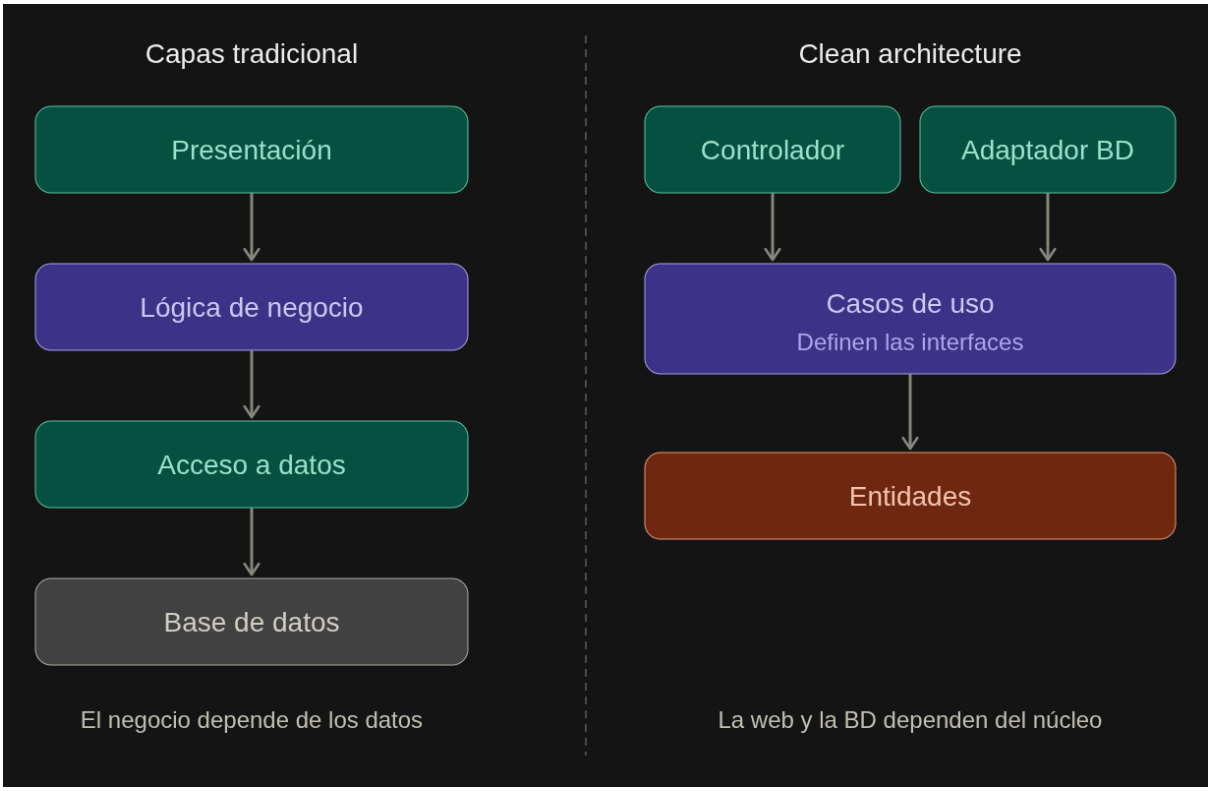
Desventajas

- Mayor complejidad inicial: Requiere escribir más código repetitivo, clases de mapeo e interfaces.
- Curva de aprendizaje: Los desarrolladores juniors pueden encontrarla excesiva o difícil de estructurar correctamente.
- Sobrecarga para proyectos pequeños: En aplicaciones CRUD sencillas o prototipos, aplicar Clean Architecture resulta incluso más laborioso.

La diferencia fundamental entre ambas se encuentra en la dirección de las dependencias y el centro de gravedad del sistema.

- En una arquitectura tradicional las capas superiores dependen de las inferiores y por lo general, la base de datos está en el centro o condiciona al diseño. Se suele decir que es una arquitectura horizontal.
- En una arquitectura limpia, el centro es el negocio puro, el cual es total y completamente independiente de los agentes externos, esto hace que las

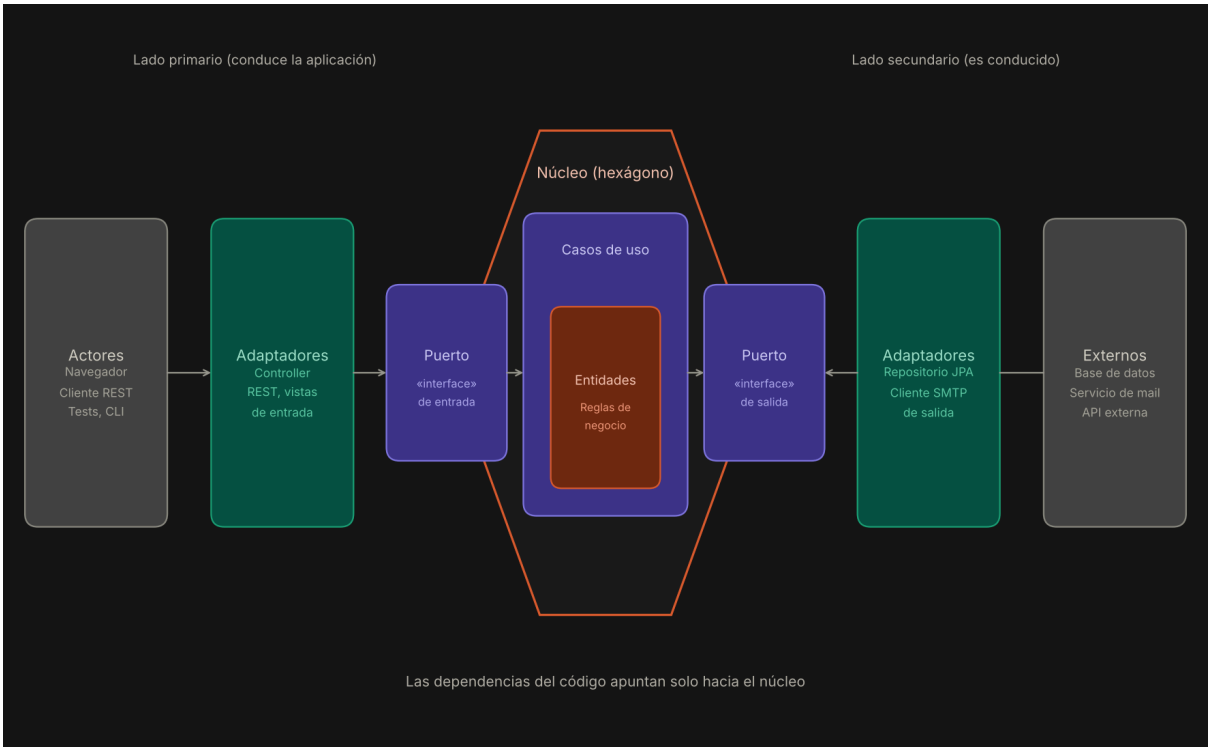
dependencias fluyan estrictamente para adentro, por lo tanto, la infraestructura depende del negocio y no al revés.



➢ ¿Qué es la arquitectura Hexagonal? Grafique la disposición de capas y explique su funcionamiento. Indique sus ventajas y desventajas. ¿Qué diferencias existen entre “clean architecture”?

La arquitectura hexagonal, también llamada arquitectura de puertos y adaptadores, es un patrón de diseño de software propuesto por Alistair Cockburn que organiza el sistema en un núcleo y un exterior, para que la lógica del negocio no dependa de la tecnología con la que se conecta. Su meta es crear sistemas que sean:

- Independientes de frameworks y librerías externas.
- Altamente comprobables de manera aislada.
- Independientes de la interfaz de usuario.
- Independientes de la base de datos.
- Capaces de ser utilizados de la misma forma por un usuario, un test o un script.



Capas:

1. Núcleo (Hexágono):
    - Está en el centro. Contiene las entidades, las reglas de negocio y los casos de uso. No conoce frameworks, bases de datos ni protocolos web.
2. Puertos:
    - Son interfaces definidas por el núcleo que establecen cómo se comunica con el exterior. Los puertos de entrada (primarios) definen lo que la aplicación ofrece, por ejemplo un caso de uso. Los puertos de salida (secundarios) definen lo que la aplicación necesita, por ejemplo un repositorio o un servicio de mail.
3. Adaptadores:
    - Son los encargados de conectar una tecnología concreta con un puerto. Los adaptadores de entrada (controladores REST, vistas, consola, tests) invocan los puertos de entrada. Los adaptadores de salida (repositorios JPA, clientes SMTP, APIs externas) implementan los puertos de salida.
4. Actores y sistemas externos:
    - Es la parte más externa, compuesta por quienes usan la aplicación (navegador, cliente REST) y por lo que la aplicación usa (base de datos, servicio de mail). Son “detalles” que se pueden reemplazar cambiando solamente el adaptador.

Ventajas

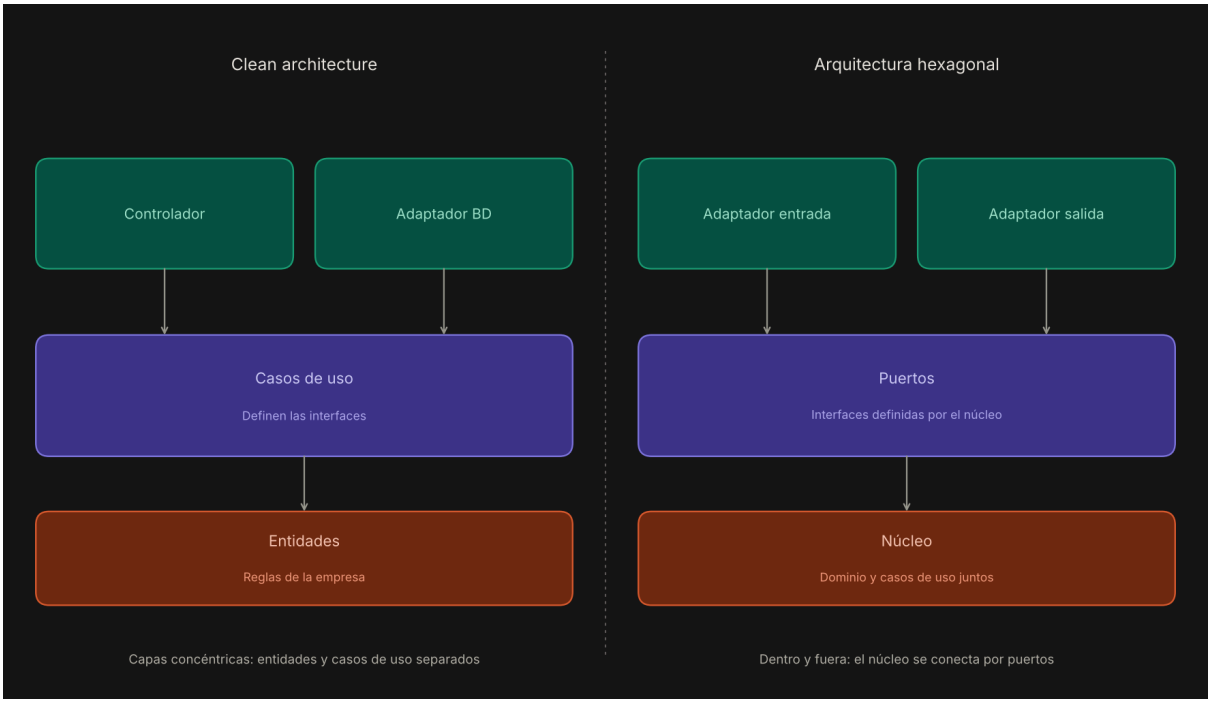
- Independencia tecnológica: Permite cambiar la base de datos, el framework web o el servicio de mail reemplazando solo el adaptador, sin tocar el núcleo.
- Testabilidad excelente: El núcleo se puede probar con adaptadores falsos (mocks o repositorios en memoria), sin levantar servidores ni bases de datos.
- Flexibilidad de entradas: La misma lógica se puede exponer por web, API, consola o tests sin duplicar código.
- Aplazamiento de decisiones: Permite empezar con un repositorio en memoria y elegir la base de datos más adelante.

Desventajas

- Mayor complejidad inicial: Requiere escribir más interfaces, adaptadores y clases de mapeo (por ejemplo, entidad de dominio y entidad JPA).
- Curva de aprendizaje: Puede resultar difícil definir qué es un puerto y dónde termina el núcleo.
- Sobrecarga para proyectos pequeños: En aplicaciones CRUD sencillas o prototipos, agrega indirección sin un beneficio claro.
- Poca guía interna: No indica cómo organizar el interior del núcleo, por lo que cada equipo lo resuelve de forma distinta.

La diferencia fundamental entre ambas se encuentra en cómo organizan el sistema, ya que las dos comparten la regla de que las dependencias apuntan hacia el negocio.

- En una arquitectura hexagonal el sistema se divide en dos zonas, dentro y fuera, y el foco está en cómo se conecta el núcleo con el exterior mediante puertos y adaptadores. Las entidades y los casos de uso se agrupan dentro de un mismo núcleo. Se suele destacar la simetría entre el lado primario (quien usa la aplicación) y el secundario (lo que la aplicación usa).
- En una arquitectura limpia el sistema se divide en cuatro capas concéntricas (entidades, casos de uso, adaptadores de interfaz y frameworks y drivers), y define con precisión qué va en cada una. Separa explícitamente las reglas de la empresa de las reglas de la aplicación, y el énfasis está en la jerarquía de capas del centro hacia afuera.


Clean architecture
Controlador
Adaptador BD
Casos de uso
Definen las interfaces
Entidades
Reglas de la empresa
Capas concéntricas: entidades y casos de uso separados
Arquitectura hexagonal
Adaptador entrada
Adaptador salida
Puertos
Interfaces definidas por el núcleo
Núcleo
Dominio y casos de uso juntos
Dentro y fuera: el núcleo se conecta por puertos